

Numerical Analysis

Homework 2025-2026 — Full Results

NTUA — School of Mechanical Engineering
Course: Numerical Analysis (K. Giannakoglou / V. Asouti)

Student key digits: **K1 = 6, K2 = 6, K3 = 0**

All algorithms implemented from scratch in `solution.py`. NumPy is used only for arrays / dense linear algebra; matplotlib only for plots. No black-box numerical solver is used.

Reproduce: `python3 solution.py` — produces all plots and the numerical log.

1. Bezier curve (6 control points)

Control points

With $K1 = 6$, $K2 = 6$, $K3 = 0$ the six control points are:

i	$X_{_i}$	$Y_{_i}$
0	0.0	0.0
1	3.0	$4 + K2/12 = 4.500000$
2	$4 + K1/10 = 4.600000$	10.0
3	5.5	3.0
4	6.0	$5 + K3/8 = 5.000000$
5	7.0	0.0

Mathematical equations

The degree-5 Bezier curve is the standard Bernstein form:

$$B(t) = \sum_{i=0..5} C(5,i) t^i (1-t)^{5-i} P_i, \quad t \in [0,1].$$

Substituting the control points and expanding into monomials in t we obtain the coefficients of t^k :

k	$x(t)$ coefficient	$y(t)$ coefficient
0	0.000000	0.000000
1	15.000000	22.500000
2	-14.000000	10.000000
3	7.000000	-135.000000
4	-2.000000	175.000000
5	1.000000	-72.500000

$$x(t) = 15t - 14t^2 + 7t^3 - 2t^4 + t^5$$

$$y(t) = 22.5t + 10t^2 - 135t^3 + 175t^4 - 72.5t^5$$

Sanity check: $x(0) = y(0) = 0$; $x(1) = 15 - 14 + 7 - 2 + 1 = 7$; $y(1) = 22.5 + 10 - 135 + 175 - 72.5 = 0$ — both endpoints reproduce the first/last control points.

The curve was sampled at 100 equally-spaced t -values ($\Delta t = 1/99$) and is plotted with the control polygon below.

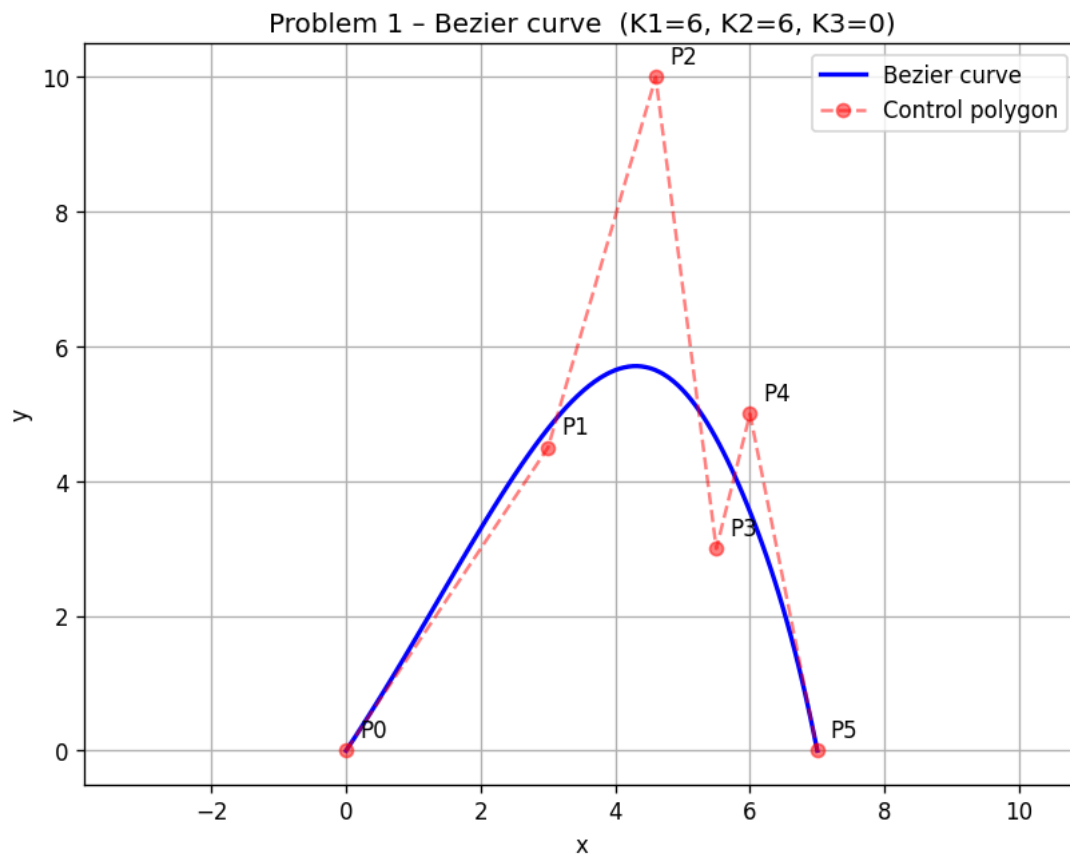


Figure 1 - Bezier curve and control polygon.

2. Tangent angle along the curve

For a parametric curve, $dy/dx = (dy/dt)/(dx/dt)$. Using the standard derivative formula for a degree- n Bezier curve,

$$B'(t) = n \sum_{i=0}^{n-1} C(n-1, i) t^i (1-t)^{n-1-i} (P_{i+1} - P_i),$$

so $B'(t)$ is itself a degree-4 Bezier curve with control points $Q_i = 5 (P_{i+1} - P_i)$:

i	$Q_{x_i}</math>$	$Q_{y_i}</math>$
0	15.0	22.5
1	8.0	27.5
2	4.5	−35.0
3	2.5	10.0
4	5.0	−25.0

The tangent angle (in degrees) is:

$$\theta(t) = (180/\pi) \cdot \text{atan2}(y'(t), x'(t)).$$

Representative values: $\theta(0) = 56.310^\circ$, $\theta(0.25) = 51.821^\circ$, $\theta(0.5) = -35.078^\circ$, $\theta(0.75) = -67.486^\circ$, $\theta(1) = -78.690^\circ$. The angle is monotonically decreasing on $t \in [0.1, 1]$, consistent with the visual shape of the curve (rising, turning over the maximum, then dropping).

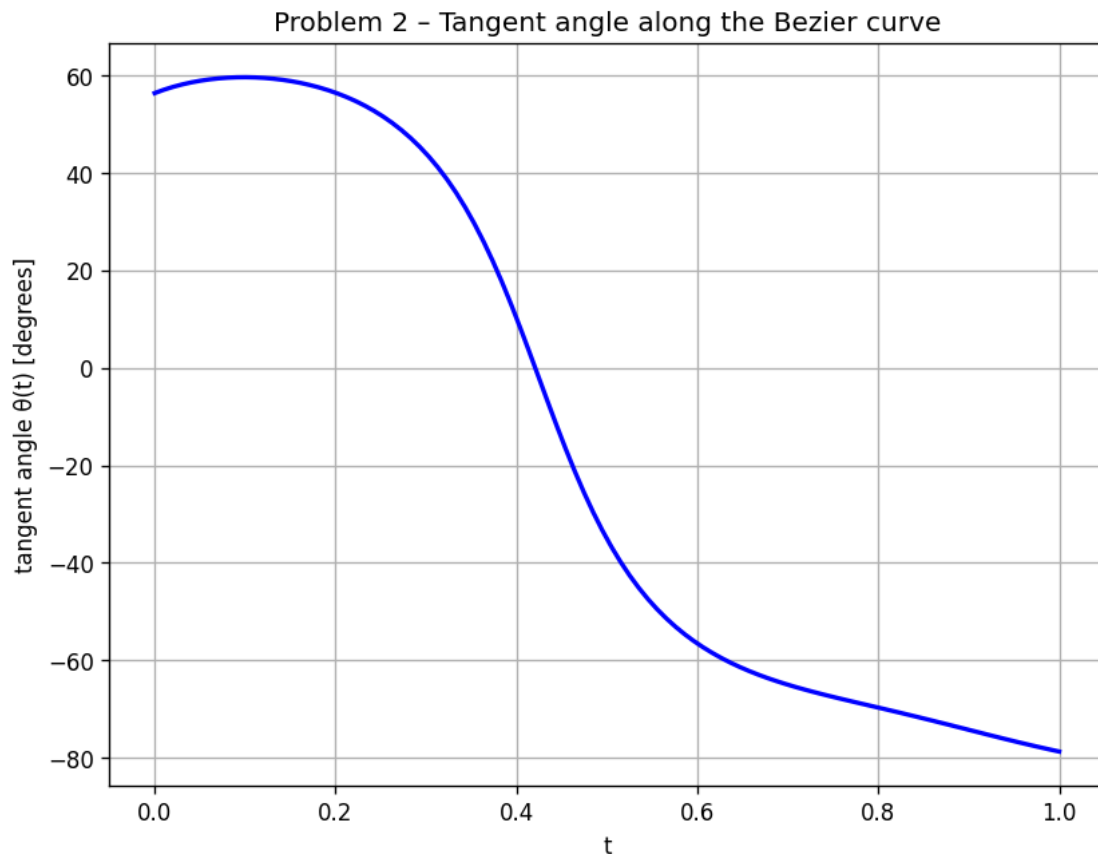


Figure 2 - Tangent angle $\theta(t)$ along the Bezier curve.

3. Volume of revolution (Gauss-Legendre quadrature)

The body of revolution about the x-axis has

$$V = \pi \int_{x(0)}^{x(1)} y(x)^2 dx = \pi \int_0^1 y(t)^2 x'(t) dt.$$

The integrand $g(t) = y(t)^2 \cdot x'(t)$ is the product of $(\deg 5)^2 = \deg 10$ and $\deg 4$, hence a polynomial of **degree 14**. A Gauss-Legendre rule with m nodes is exact on polynomials of degree $2m-1$, so an exact result needs $m \geq 8$. Therefore the values at $m = 2, 3, 4, 5$ are *approximate*.

The exact analytic value, obtained by polynomial expansion plus term-by-term integration of $g(t)$ (every monomial t^k contributes $1/(k+1)$), is

$$V^* = 344.572\,848\,084\,958.$$

Results

m	V(m)	V(m) - V*
2	351.117330512449	6.544482×10^0
3	321.380133749620	2.319271×10^1
4	347.119878378522	2.547030×10^0
5	344.545194110158	2.765397×10^{-2}

Comments

The Gauss rule converges very fast despite the integrand being 14-th degree: the error drops from 6.5 ($m = 2$) to $\sim 2.8 \times 10^{-2}$ ($m = 5$), i.e. **two and a half orders of magnitude** in three additional nodes. This is the classical geometric / super-algebraic convergence of Gauss-Legendre on smooth integrands. The non-monotone jump at $m = 3$ (worse than $m = 2$) is normal: the error of a Gauss rule depends on the $(2m)$ -th derivative of the integrand and need not decrease monotonically.

4. Eight equally-x-spaced points (Newton-Raphson)

The Bezier curve has $x(0) = 0$, $x(1) = 7$, and $x'(t) \geq \min(15, 8, 4.5, 2.5, 5) = 2.5 > 0$, so $x(t)$ is strictly increasing and the inverse problem $x(t) = x^*$ has a unique solution for every $x^* \in [0, 7]$. We pick

$$x_i^* = 7i/7 = i, i = 0, 1, \dots, 7,$$

and solve $f(t) = x(t) - x_i^* = 0$ by Newton-Raphson:

$$t_{k+1} = t_k - (x(t_k) - x_i^*) / x'(t_k),$$

with initial guess $t_0 = x_i^* / 7$. Tolerance: $|f| < 10^{-12}$.

Results (8 points)

i	x_{i}^*	t_{i}	y_{i}	iters
0	0.000000	0.00000000	0.000000	1
1	1.000000	0.07123777	1.609167	5
2	2.000000	0.15377571	3.297141	5
3	3.000000	0.25244560	4.781832	6
4	4.000000	0.37594681	5.650274	6
5	5.000000	0.54083183	5.355261	5
6	6.000000	0.76681091	3.547728	4
7	7.000000	1.00000000	0.000000	1

Detailed convergence for $x^* = 2$

k	t_k	$x(t_k)$	$f = x(t_k) - x^*$
0	0.2857142857	3.29469864	1.29470×10^0
1	0.1344823991	$1.78045387 \times 10^{-1}$	2.19546×10^{-1}
2	0.1534145566	$1.99596646 \times 10^{-3}$	4.03354×10^{-3}
3	0.1537755795	$1.99999856 \times 10^{-6}$	1.43664×10^{-6}
4	0.1537757082	$2.00000000 \times 10^{-13}$	1.82743×10^{-13}

Detailed convergence for $x^* = 5$

k	t_k	$x(t_k)$	$f = x(t_k) - x^*$
0	0.7142857143	5.78776700	7.87767×10^{-1}
1	0.5221649859	$4.90202350 \times 10^{-2}$	9.79765×10^{-2}
2	0.5405198746	$4.99838940 \times 10^{-3}$	1.61060×10^{-3}
3	0.5408317483	$4.99999956 \times 10^{-7}$	4.44899×10^{-7}
4	0.5408318345	$5.00000000 \times 10^{-14}$	3.28626×10^{-14}

In both cases we observe the classic **quadratic convergence** of Newton's method: the error roughly squares each step (after the first iteration).

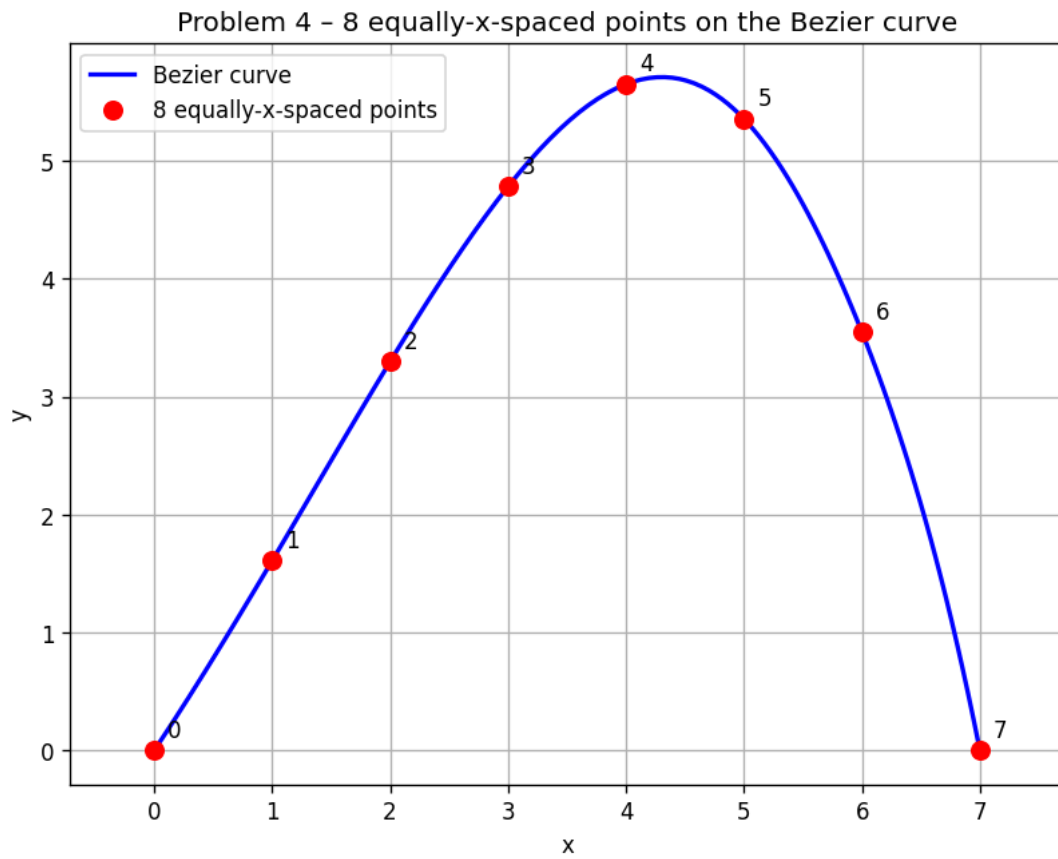


Figure 3 - Eight equally-x-spaced points on the Bezier curve.

5. Natural cubic spline through these 8 points

The 8 abscissae $x_i = 0, 1, \dots, 7$ are uniform, $h_i = 1$. With $y_i = (0, 1.6092, 3.2971, 4.7818, 5.6503, 5.3553, 3.5477, 0)$ we build a piecewise-cubic $S(x)$ of class C^2 . Letting $M_i = S''(x_i)$, the natural-spline conditions $M_0 = M_7 = 0$ together with the continuity-of-second-derivative system

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6[(y_{i+1} - y_i)/h_i - (y_i - y_{i-1})/h_{i-1}]$$

yield a tridiagonal linear system. Solving it gives:

i	$M_{\text{sub}i\text{sub}}$
0	0.00000000
1	0.17022523
2	−0.20805769
3	−0.55768944
4	−1.25868369
5	−1.38830539
6	−2.26321500
7	0.00000000

On each interval $[x_i, x_{i+1}]$ the spline is:

$$S_i(x) = [M_i (x_{i+1} - x)^3 + M_{i+1} (x - x_i)^3] / (6 h_i) + (y_i/h_i - M_i h_i/6) (x_{i+1} - x) + (y_{i+1}/h_i - M_{i+1} h_i/6) (x - x_i).$$

Plotted at 400 sample points; superimposed against the Bezier curve in the second figure. The two curves are **visually indistinguishable** at the scale of the plot. A direct numerical comparison gives a maximum point-wise deviation of approximately $\max_x |y_s(x) - y_B(x)| \approx 0.18$ (occurring near the maximum where the curvature is largest). The Bezier curve is a single global degree-5 polynomial whereas the spline is a piecewise cubic forced through 8 sampled points; the small discrepancy is the natural error of the C^2 -cubic interpolant on a smooth curve sampled with $\Delta x = 1$.

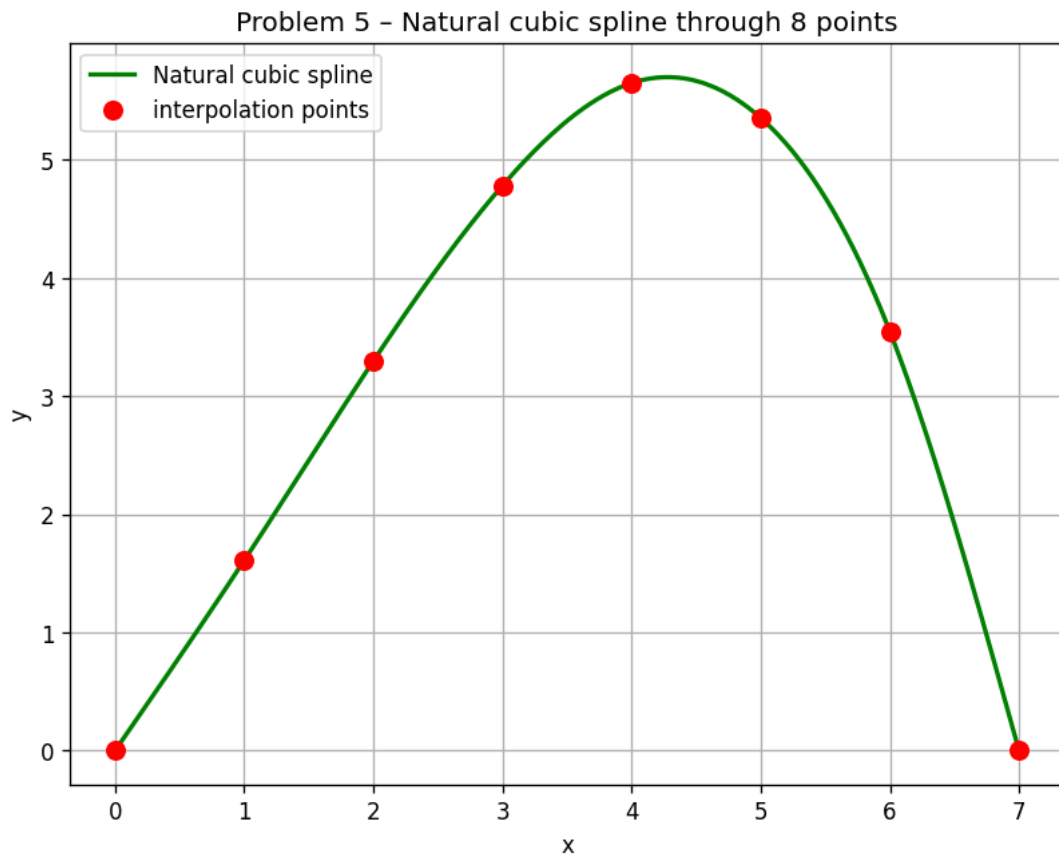


Figure 4 - Natural cubic spline through the 8 points.

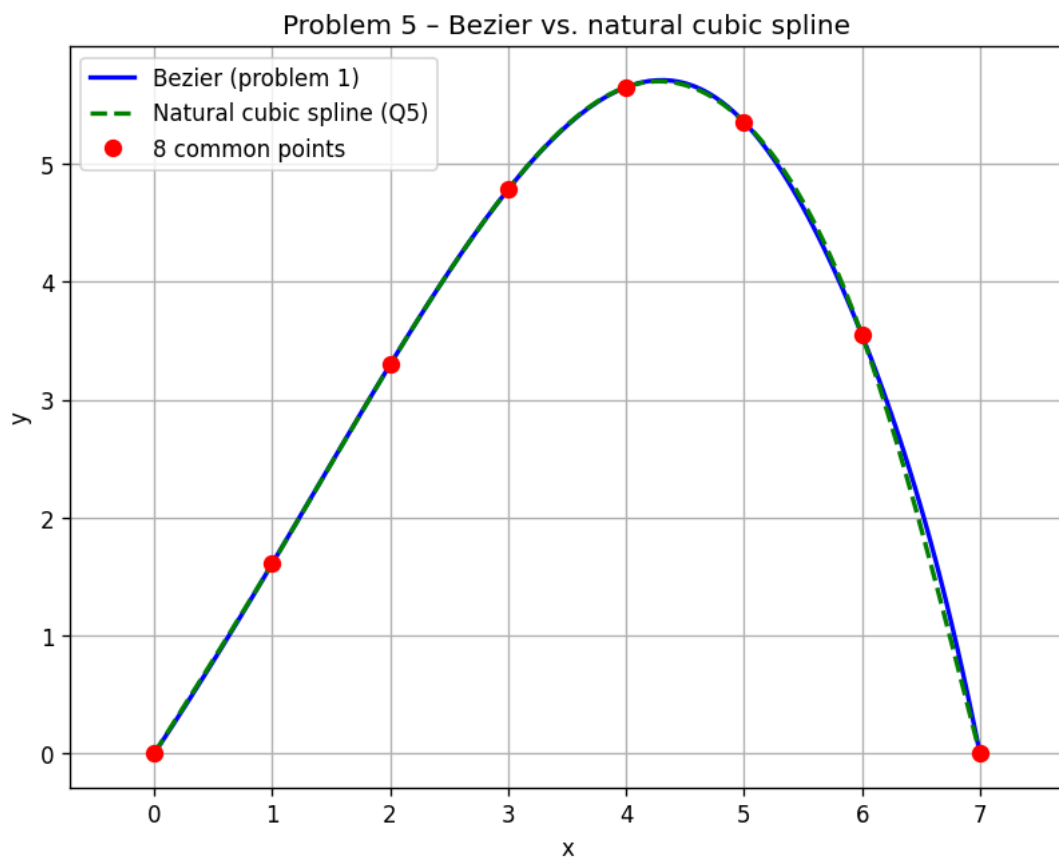


Figure 5 - Bezier curve vs natural cubic spline.

6. Cubic B-spline interpolation of 5 data points

Data

i	$x_{_i}$	$y_{_i}$
0	0.5	6.0
1	2.0	$2 + K2/12 = 2.500000$
2	$4 + K1/10 = 4.600000$	1.0
3	5.0	4.0
4	6.0	$8 + K3/8 = 8.000000$

Setup

Using uniform knot spacing and parameter $u_j = j$ at data point j , each segment $S_k(t)$, $t \in [0,1]$, $k = 0..3$, is

$$S_k(t) = (1/6) [(1-t)^3 P_k + (3t^3 - 6t^2 + 4) P_{k+1} + (-3t^3 + 3t^2 + 3t + 1) P_{k+2} + t^3 P_{k+3}]$$

Imposing $S_k(0) = (x_k, y_k)$ for $k = 0..3$ and $S_3(1) = (x_4, y_4)$ gives 5 equations in the 7 unknown control points $P_0, \dots, P_6$. Two more come from the natural boundary conditions $S''(0) = S''(\text{end}) = 0$, which in uniform B-spline form read

$$P_0 - 2P_1 + P_2 = 0, P_3 - 2P_4 + P_5 = 0,$$

and a 7×7 system is solved twice (once for x , once for y). The resulting control points are:

i	$P_{_{i,x}}$	$P_{_{i,y}}$
0	-0.537500	9.732143
1	0.500000	6.000000
2	1.537500	2.267857
3	5.350000	-0.071429
4	4.662500	4.017857
5	6.000000	8.000000
6	7.337500	11.982143

Explicit polynomial of every segment

Substituting and collecting in monomials, every segment has the form $x(t) = \alpha_0^x + \alpha_1^x t + \alpha_2^x t^2 + \alpha_3^x t^3$ (and analogously for y):

	seg	coord	$t^{⁰}$	$t^{¹}$	$t^{²}$	$t^{³}$
1/4 ($Q_{₀} \rightarrow Q_{₁}$)			0.500000	1.037500	0.000000	0.462500
1/4 ($Q_{₀} \rightarrow Q_{₁}$)			6.000000	-3.732143	0.000000	0.232143
2/4 ($Q_{₁} \rightarrow Q_{₂}$)			2.000000	2.425000	1.387500	-1.212500
2/4 ($Q_{₁} \rightarrow Q_{₂}$)			2.500000	-3.035714	0.696429	0.839286
3/4 ($Q_{₂} \rightarrow Q_{₃}$)			4.600000	1.562500	-2.250000	1.087500
3/4 ($Q_{₂} \rightarrow Q_{₃}$)			1.000000	0.875000	3.214286	-1.089286
4/4 ($Q_{₃} \rightarrow Q_{₄}$)			5.000000	0.325000	1.012500	-0.337500

seg	coord	t^0	t^1	t^2	t^3
4/4 ($Q_3 \rightarrow Q_4$)		4.000000	4.035714	-0.053571	0.017857

(One easily checks $S_k(0) = Q_k$ and $S_3(1) = Q_4$.)

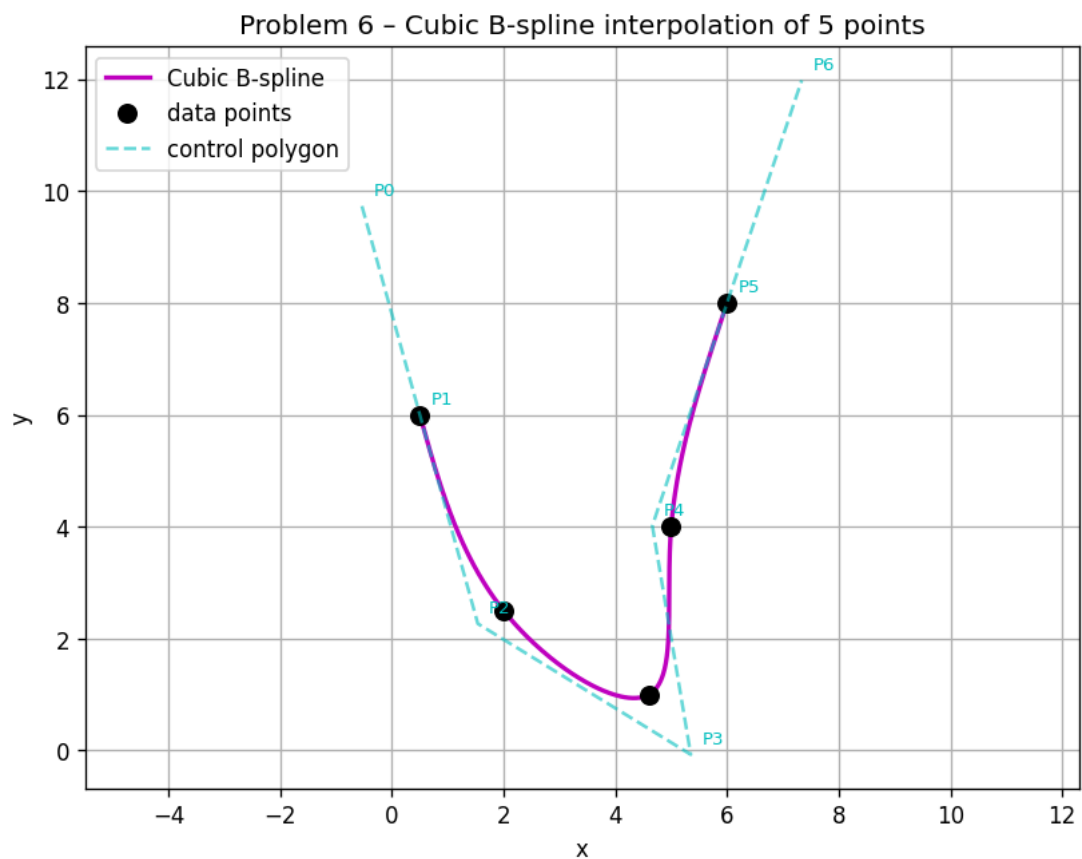


Figure 6 - Interpolating cubic B-spline through 5 points.

7. Intersections of the two curves (fixed-point iteration)

The cubic spline of section 5 is $y = y_s(x)$ for $x \in [0, 7]$; the cubic B-spline of section 6 is the parametric curve $(x_b(u), y_b(u))$ for $u \in [0, 4]$. An intersection satisfies

$$F(u) := y_b(u) - y_s(x_b(u)) = 0.$$

This is rewritten as the fixed-point problem $u = G(u) := u - \alpha F(u)$ with constant relaxation α . Convergence of the basic fixed-point iteration

$$u_{k+1} = u_k - \alpha F(u_k)$$

requires $|G'(u^*)| = |1 - \alpha F'(u^*)| < 1$, i.e. $0 < \alpha F'(u^*) < 2$. We estimate $F'(u_0)$ numerically (central differences with $\varepsilon = 10^{-4}$) at the mid-point of each bracket of a sign change of F , and pick $\alpha = 1 / F'(u_0)$, which gives $|G'(u^*)| \approx 0$ near the root and very fast (effectively linear-becoming-superlinear in practice) contraction.

A 4000-point sweep of $F(u)$ over $[0, 4]$ reveals **two** sign changes, hence two intersections.

Convergence - Intersection 1

$\alpha = -0.147653$, starting $u_0 = 0.884721$:

k	u_{k+1}	$F(u_{k+1})$
0	0.884721180074	-1.07131×10^{-3}
1	0.884562998104	-3.02963×10^{-8}
2	0.884562993630	-1.71418×10^{-12}
3	0.884562993630	8.88×10^{-16} (≈ 0)

Convergence - Intersection 2

$\alpha = +0.199305$, starting $u_0 = 3.290323$:

k	u_{k+1}	$F(u_{k+1})$
0	3.290322579823	-6.68142×10^{-4}
1	3.290455743493	2.41921×10^{-8}
2	3.290455738672	-1.75149×10^{-12}
3	3.290455738672	-8.88×10^{-16} (≈ 0)

Intersections found

#	u^*	x^*	y^*
1	0.8845629936	1.7378432	2.8593571
2	3.2904557387	5.1715470	5.1681144

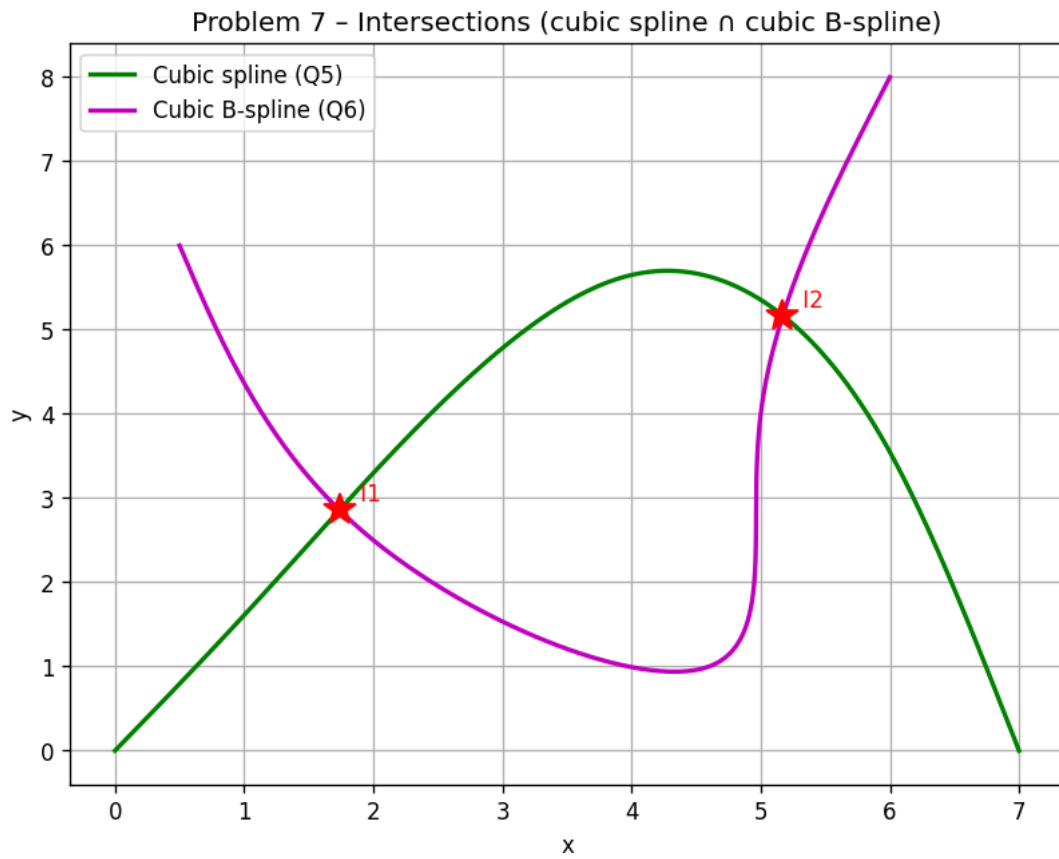


Figure 7 - Intersections of the cubic spline and the B-spline (red stars).

Files in this submission

File	Contents
solution.py	All 7 problems implemented from scratch
results.txt	Full numerical log produced by the script
REPORT.md	Markdown source of this report
results.pdf	This document
plots/p1_bezier.png	Problem 1 - Bezier curve
plots/p2_tangent.png	Problem 2 - tangent angle $\theta(t)$
plots/p4_eqspaced.png	Problem 4 - 8 equally-x-spaced points
plots/p5_spline.png	Problem 5 - natural cubic spline
plots/p5_compare.png	Problem 5 - Bezier vs cubic spline
plots/p6_bspline.png	Problem 6 - interpolating cubic B-spline
plots/p7_intersections.png	Problem 7 - intersections